# Fake News Detection using Machine Learning

A plain-English study guide: what the project does, how the data flows, and what every cell of the notebook is for.

**Kushal Shah** | C064
**Varun Pitale** | C080
**Kanishk Modi** | C119

Based on the open-source notebook by kapilsinghnegi (github.com/kapilsinghnegi/Fake-News-Detection). All figures marked "our re-run" were recomputed by us on the same dataset.

# 1. The big picture

Fake news spreads fast online. This project builds a program that reads the text of a news article and predicts one of two labels: **Fake** or **Not fake (real)**. Nobody writes rules by hand. Instead we show the computer tens of thousands of articles that are already labelled, and it works out which patterns of words tend to go with each label.

In machine learning terms this is **supervised learning** (the examples come with answers) and, more specifically, **binary text classification** (two possible answers, input is text).

**The whole project in one sentence**

Load labelled articles, clean the text, turn words into numbers, train four classifiers on 75% of the data, measure them on the other 25%, then let them vote on any new article you paste in.

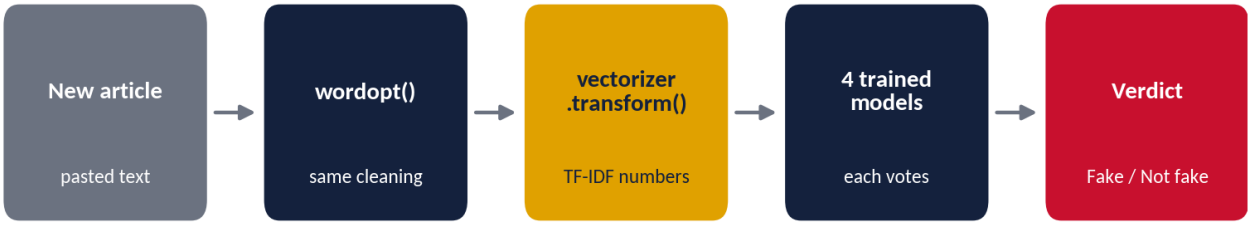



Figure 1. What happens when you paste a new article (explained in Section 9).

## What you will learn from this guide

- what the dataset contains and how it is prepared (Sections 3 and 4),
- what every block of the notebook does and why it is there (Section 5),
- how TF-IDF and the four models work, in simple terms (Sections 6 and 7),
- how to read the results, and what the 99% accuracy really means (Sections 8 and 10),
- questions a teacher or examiner is likely to ask, with answers (Section 12).

# 2. Key terms in simple words

| Term | Meaning in this project |
|---|---|
| Machine learning | Letting a program find patterns in examples instead of writing the rules ourselves. |
| Supervised learning | Every training example has a known answer (its label), so the program can check itself while learning. |
| Classification | Predicting a category. Here: Fake (0) or Real (1). |
| NLP (natural language processing) | Techniques for working with human text. Cleaning and TF-IDF are NLP steps. |
| Feature | A measurable property the model uses. Here, each word in the vocabulary is one feature (about 95,000 of them in our re-run). |
| Vectorization / TF-IDF | Converting text into numbers. Each article becomes a long list of word scores that a model can do maths on. |
| Training set / test set | Training articles are used to learn. Test articles are kept hidden until the end to measure honestly. Split: 75% / 25%. |
| Model / classifier | The learned program that takes an article's numbers and outputs 0 or 1. |
| Accuracy | Share of test articles classified correctly. |
| Precision | Of the articles the model labelled as a class, how many really belonged to it. |
| Recall | Of the articles that really belong to a class, how many the model found. |
| F1-score | One number that balances precision and recall. |
| Confusion matrix | A 2 x 2 table counting correct and wrong predictions for each class. |
| Overfitting | Memorising the training data so well that new data is handled badly. |
| Data leakage / shortcut | When the data contains an accidental giveaway (like a source name) so the model scores high without learning the real task. |

# 3. The dataset

The project uses two CSV files, `Fake.csv` and `True.csv`. They appear to be the widely used ISOT Fake News dataset: English news articles from roughly 2015 to 2018, mostly about US politics and world affairs. The real articles were collected from Reuters and the fake ones from sites flagged as unreliable. (This is our reading of the files; the notebook itself does not state the source.)

| Column | What it holds | Used for training? |
|---|---|---|
| title | Headline of the article | No (dropped) |
| text | Full body text of the article | **Yes** - the only input |
| subject | Topic or section label (for example politics, worldnews) | No (dropped) |
| date | Publication date | No (dropped) |
| class | Label added by the notebook: 0 = Fake, 1 = Real | **Yes** - the answer to predict |

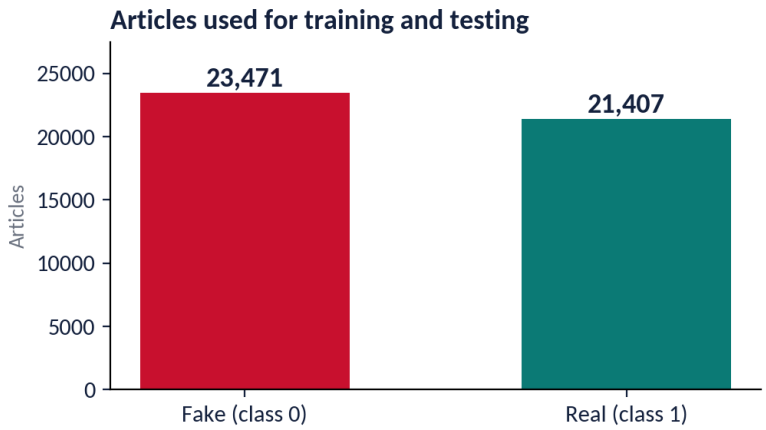



Figure 2. After holding back 10 rows per file, 44,878 articles remain. The classes are fairly balanced.

## Facts we measured on the data (our re-run)

| Item | Fake.csv | True.csv |
|---|---|---|
| Articles in the file | 23,481 | 21,417 |
| Median article length | about 363 words | about 359 words |
| Articles with empty body text | 630 | 1 |
| Values in "subject" | News, politics, left-news, Government News, US_News, Middle-east | politicsNews, worldnews |
| Articles that mention "Reuters" | 1.4% | 99.8% |

**Two things to notice**

First, the "subject" values never overlap between the two files, so that column alone would label every article perfectly. It is a giveaway, which is why the notebook correctly drops it. Second, almost every real article contains the word "Reuters". We come back to this in Section 10 because it explains a lot about how the model behaves.

# 4. Data flow, start to finish

The diagram shows how raw files become a prediction. Every box is a block of notebook code, explained in the next section.

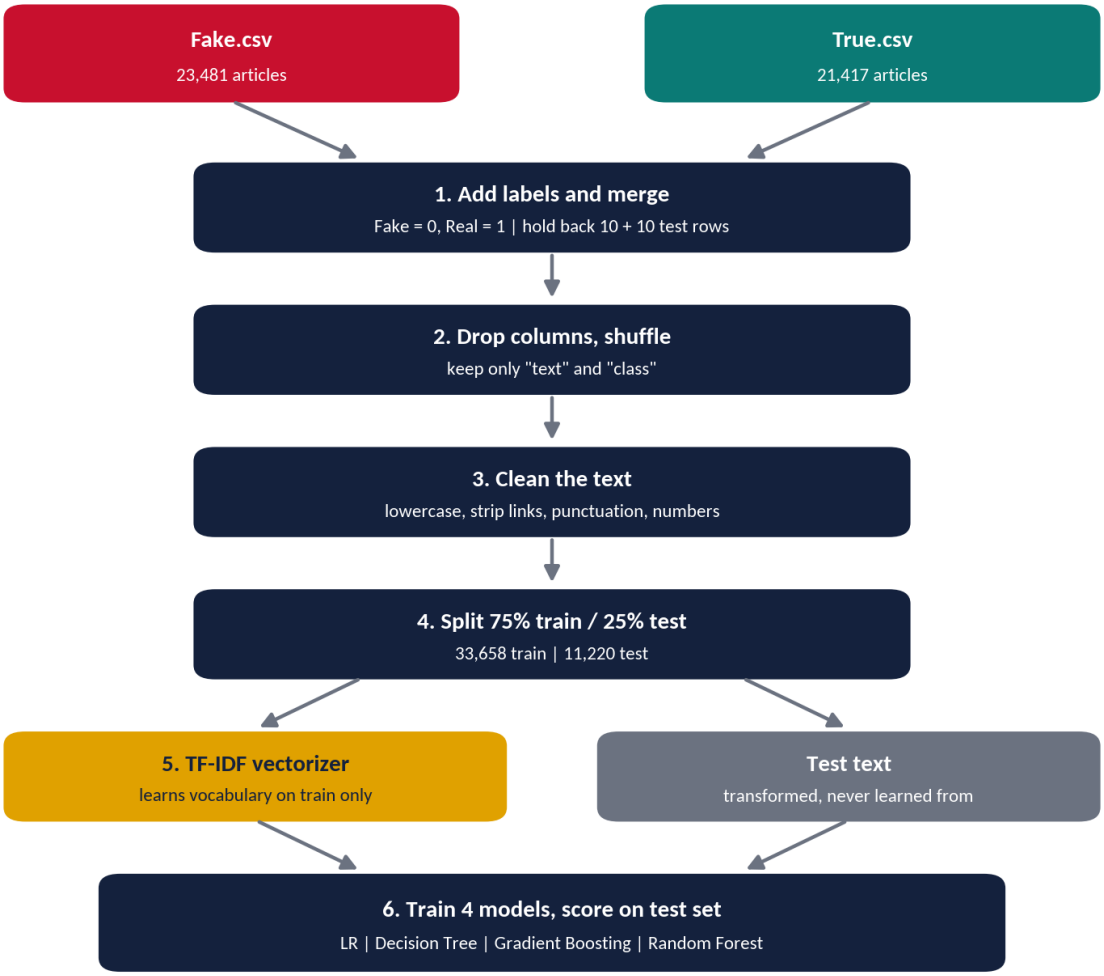



Figure 3. End-to-end data flow of the training pipeline.

### Why the vectorizer only learns from training text

The TF-IDF vectorizer builds its vocabulary and word weights from the training articles only. Test articles are transformed using that frozen vocabulary. If the test set influenced the vocabulary, the test would no longer be a fair "unseen data" check.

# 5. The notebook, block by block

The notebook has about 65 cells. We group them by the section headings in the notebook itself, so you can follow along in Colab. For each block: the code, what it does, and why it is there.

## Step 1. Import libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import string
import re
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
```

- **pandas** loads and manipulates tables (called DataFrames).
- **re** (regular expressions) and **string** are used for text cleaning.
- **train_test_split** divides the data into training and test parts.
- **classification_report** prints precision, recall and F1.
- matplotlib, numpy, seaborn and accuracy_score are imported but not actually used later; they are harmless leftovers.

## Step 2. Load the data and preview it

```
data_fake = pd.read_csv('Fake.csv')
data_true = pd.read_csv('True.csv')
data_fake.head()      # first 5 rows
data_true.tail()      # last 5 rows
```

Each CSV becomes a DataFrame with the columns title, text, subject and date. `head()` and `tail()` only look at the first or last rows so you can sanity-check what was loaded. This is also the cell that fails with FileNotFoundError if the CSVs are not in the working folder.

## Step 3. Add labels

```
data_fake["class"] = 0
data_true["class"] = 1
data_fake.shape, data_true.shape     # ((23481, 5), (21417, 5))
```

Models need the correct answer next to every example. A new column **class** is added: 0 for every fake article, 1 for every real one. `shape` shows (rows, columns); the 5 columns are title, text, subject, date and class.

## Step 4. Hold back rows for manual testing

```
data_fake_manual_testing = data_fake.tail(10)
for i in range(23480, 23470, -1):
    data_fake.drop([i], axis=0, inplace=True)

data_true_manual_testing = data_true.tail(10)
for i in range(21416, 21406, -1):
    data_true.drop([i], axis=0, inplace=True)
```

The last 10 articles of each file are copied into separate "manual testing" tables and then removed from the main data. The model therefore never trains on them, so they are 20 fresh articles you could try by hand. After this the shapes become (23471, 5) and (21407, 5). Two details: Pandas prints a SettingWithCopyWarning when the label is added to the copied rows (harmless here), and in this notebook those held-back tables are displayed but never fed to the models.

## Step 5. Merge, drop columns, check for missing values

```
data_merge = pd.concat([data_fake, data_true], axis=0)
data = data_merge.drop(['title', 'subject', 'date'], axis=1)
data.isnull().sum()
```

`concat(axis=0)` stacks the two tables on top of each other, giving one table of 44,878 rows. Then title, subject and date are dropped so only **text** and **class** remain. The notebook's own comment says these columns are not needed. Dropping **subject** is genuinely important because it would leak the answer (Section 3). `isnull().sum()` confirms there are no missing values (0 and 0). Note that some texts are empty strings, which is different from missing.

## Step 6. Shuffle

```
data = data.sample(frac=1)
data.reset_index(inplace=True)
data.drop(['index'], axis=1, inplace=True)
```

After the merge, all fake rows come first and all real rows come last. If we split the data now, the test set could be only real news. `sample(frac=1)` returns 100% of the rows in random order. The next two lines renumber the rows 0, 1, 2… and remove the old index column.

## Step 7. Clean the text with wordopt()

```
def wordopt(text):
    text = text.lower()
    text = re.sub('\[.*?\]', '', text)
    text = re.sub("\\W", " ", text)
    text = re.sub('https?://\S+|www\.\S+', '', text)
    text = re.sub('<.*?>+', b'', text)
    text = re.sub('[%s]' % re.escape(string.punctuation), '', text)
    text = re.sub('\w*\d\w*', '', text)
    return text

data['text'] = data['text'].apply(wordopt)
```

| Line | Effect |
|---|---|
| lower() | Everything lowercase, so "Trump" and "trump" count as the same word. |
| remove [ … ] | Deletes anything inside square brackets, such as [video] or editor notes. |

| Line | Effect |
|---|---|
| replace \W with space | Every non-letter, non-digit, non-underscore character (punctuation, quotes, symbols) becomes a space. |
| remove URLs | Deletes http(s):// and www. links. |
| remove | Meant to delete HTML tags. In practice the line before already removed all "<" and ">", so it never matches. |
| remove punctuation | Removes any remaining punctuation characters. |
| remove words with digits | Deletes any token containing a number (such as "2016" or "covid19"). |

**Example from our re-run.** Raw: *"WASHINGTON (Reuters) - Trump campaign adviser George Papadopoulos told an Australian diplomat in May 2016 that Russia had political dirt..."* Cleaned: *"washington reuters trump campaign adviser george papadopoulos told an australian diplomat in may that russia had political dirt..."*. Notice "(Reuters)" survives as the plain word "reuters" and "2016" disappears. `apply(wordopt)` runs the function on every article.

## Step 8. Define x and y, split into train and test

```
x = data['text']      # input: what the model reads
y = data['class']     # output: what the model must predict
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.25)
```

By convention **x** is the input and **y** the answer. `test_size=0.25` keeps 25% aside: 33,658 articles for training and 11,220 for testing. No `random_state` is given, so the split is different on every run, which is why accuracy changes slightly each time (our re-run: 98.63% for Logistic Regression versus 98.78% in the notebook).

## Step 9. Turn text into numbers (TF-IDF)

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorization = TfidfVectorizer()
xv_train = vectorization.fit_transform(x_train)
xv_test  = vectorization.transform(x_test)
```

Models cannot read words, only numbers. `fit_transform` on the training text does two things: **fit** learns the vocabulary (95,065 distinct words in our re-run) and the importance of each word; **transform** converts every article into a row of 95,065 numbers. On the test text we call only `transform`, reusing the vocabulary learned from training. Section 6 explains TF-IDF in detail.

## Step 10. Train and evaluate the four models

Each model follows the same four-line pattern:

```
LR = LogisticRegression()
LR.fit(xv_train, y_train)          # 1. learn from training data
pred_lr = LR.predict(xv_test)      # 2. predict the unseen test articles
LR.score(xv_test, y_test)          # 3. accuracy = share predicted correctly
print(classification_report(y_test, pred_lr))   # 4. precision / recall / F1
```

- **Logistic Regression:** `LogisticRegression()`
- **Decision Tree:** `DecisionTreeClassifier()`

- **Gradient Boosting:** `GradientBoostingClassifier(random_state=0)`
- **Random Forest:** `RandomForestClassifier(random_state=0)`

**A small bug in the notebook**

In the Decision Tree section the report line is `classification_report(y_test, pred_lr)`. It should use `pred_dt`. As written, it prints the Logistic Regression report again. The Decision Tree accuracy from `DT.score()` (99.53%) is still correct. Fix: change pred_lr to pred_dt in that cell.

## Step 11. Test with a new article

```
def output_lable(n):
    if n == 0:  return "Fake News"
    elif n == 1: return "Not A Fake News"

def manual_testing(news):
    testing_news = {"text": [news]}
    new_def_test = pd.DataFrame(testing_news)
    new_def_test['text'] = new_def_test["text"].apply(wordopt)
    new_xv_test = vectorization.transform(new_def_test["text"])
    pred_LR = LR.predict(new_xv_test)
    pred_DT = DT.predict(new_xv_test)
    pred_GB = GB.predict(new_xv_test)
    pred_RF = RF.predict(new_xv_test)
    return print(...)    # shows all four verdicts

news = str(input())
manual_testing(news)
```

**output_lable** turns the model's 0 or 1 back into readable text. **manual_testing** repeats the exact training recipe on one new article: put it in a one-row table, clean it with wordopt, convert it with the **same** fitted vectorizer, then ask all four models. `input()` makes Colab show a text box under the cell where you paste an article. Section 9 walks through it.

# 6. TF-IDF: how words become numbers

TF-IDF stands for **Term Frequency - Inverse Document Frequency**. It scores every word in every article by how useful it is for telling articles apart.

## The two parts

- **Term frequency (TF):** how often the word appears in this article. A word used many times probably matters to this article.
- **Inverse document frequency (IDF):** how rare the word is across all articles. Words found everywhere ("the", "is") get a low score; words found in few articles get a high score.

**Score = TF x IDF.** A word scores high when it is frequent in this article but rare overall. scikit-learn also normalises each article's vector to length 1, so long and short articles are comparable.

| Word | Appears in this article | Appears across all articles | TF-IDF score |
|---|---|---|---|
| the | many times | almost every article | low (common, tells us little) |
| reuters | once or twice | almost every real article, almost no fake ones | informative for the model |
| papadopoulos | several times | very few articles | high (distinctive) |

## What the model actually receives

Each article becomes a vector of about 95,000 numbers, one per vocabulary word. Because any article uses only a few hundred of them, nearly all entries are zero. This is called a **sparse matrix**, and scikit-learn stores it efficiently. The training matrix has 33,658 rows (articles) and about 95,000 columns (words).

Limitation: TF-IDF ignores word order. "Dog bites man" and "man bites dog" look identical. It captures vocabulary, not meaning.

# 7. The four models, simply explained

All four receive the same TF-IDF numbers and the same labels. They differ in how they turn those numbers into a decision.

| Model | Idea in plain words | Strengths | Weaknesses |
|---|---|---|---|
| **Logistic Regression** | Gives every word a weight. Words with positive weights push towards Real, negative towards Fake. It adds up the weights for the article and converts the total into a probability. | Very fast (under a second here). Easy to inspect: you can list the words with the largest weights. | Only draws a straight boundary; cannot capture word interactions. |
| **Decision Tree** | Asks a chain of yes/no questions about word scores, such as "is the score of 'reuters' above some value?", until it reaches a label. | Easy to picture and explain. | Can memorise the training data (overfit). Slow to train on 95,000 features. |
| **Gradient Boosting** | Builds many small trees one after another. Each new tree focuses on the articles the previous trees got wrong. | Usually very accurate. | Slowest to train; many settings to tune. |
| **Random Forest** | Builds many trees on random slices of the data and features, then takes a majority vote. | Stable and less prone to overfitting than a single tree. | Larger and slower than Logistic Regression; less interpretable. |

## Which words did Logistic Regression rely on? (our re-run)

Reading the largest positive and negative weights shows what the model learned:

| Pushes towards REAL | Pushes towards FAKE |
|---|---|
| reuters, said, on, washington, in, republican, wednesday, thursday, tuesday, minister, told, had, friday, nov, monday | via, us, this, read, mr, that, just, com, gop, image, featured, is, hillary, america, obama |

These are mostly **style and source markers**, not facts: wire-service wording ("reuters", "said", weekday names) versus blog-style wording ("via", "read", "image", "featured"). That is the key to understanding the results.

# 8. Results and how to read them

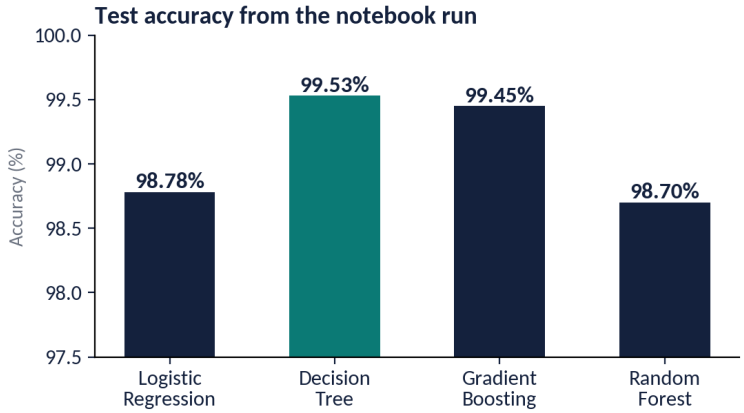



Figure 4. Accuracy of each model as printed in the notebook (test set of 11,220 articles).

| Model | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|
| Logistic Regression | 98.78% | 0.99 | 0.99 | 0.99 |
| Decision Tree | 99.53% | 0.99* | 0.99* | 0.99* |
| Gradient Boosting | 99.45% | 0.99 | 0.99 | 0.99 |
| Random Forest | 98.70% | 0.99 | 0.99 | 0.99 |

*The notebook prints the Logistic Regression report in the Decision Tree section (the bug in Step 10), so the Decision Tree row shows the same 0.99 values; its accuracy comes from `DT.score()`.

## Reading the classification report

The test set has 5,834 fake and 5,386 real articles (the "support" column). Precision, recall and F1 of 0.99 mean the models rarely mislabel either class. The four accuracies are within about one percentage point of each other. Because the split is random and unseeded, the ranking can change between runs, so do not read too much into who is "best".

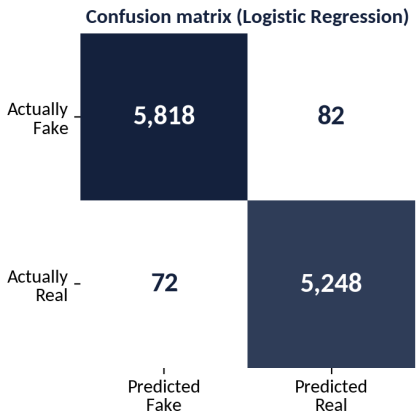
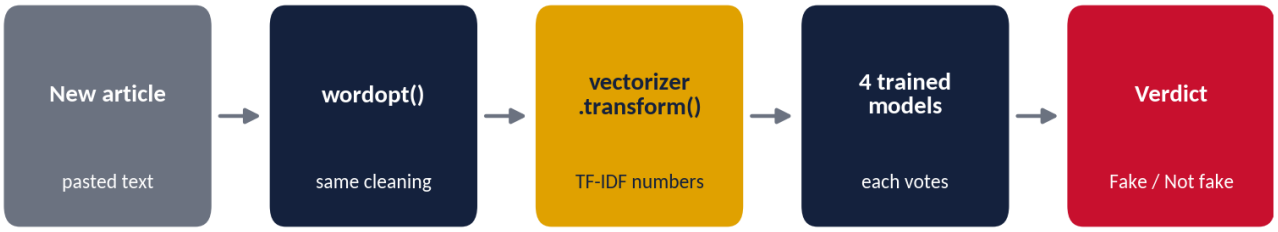



Figure 5. Confusion matrix, Logistic Regression (our re-run with a fixed random split, 98.63% accuracy).

How to read the confusion matrix: rows are the truth, columns are the prediction. 5,818 fake articles were correctly called fake and 5,248 real articles were correctly called real. 82 fake articles were wrongly called real, and 72 real articles were wrongly called fake. The mistakes are small and balanced.

# 9. How a new article is checked




Follow one article through manual_testing():

| Stage | What happens | Why |
|---|---|---|
| 1. Input | You paste an article into the input() box. It is stored as a string. | This is the unseen data. |
| 2. Wrap | The text is placed in a one-row table with a "text" column. | The cleaning code expects a table column. |
| 3. wordopt() | Lowercase, remove links, punctuation and numbers. | It must be treated exactly like the training articles. |
| 4. transform() | The fitted TF-IDF vectorizer converts the text to a row of numbers. Words never seen in training are ignored. | Models only understand numbers, and must use the same vocabulary as in training. |
| 5. predict() | Each of the four models outputs 0 or 1. | Each has its own way of deciding. |
| 6. Label | output_lable() maps 0 to "Fake News" and 1 to "Not A Fake News". | Human-readable result. |

**The verdict is not a fact-check**

The models do not know whether anything in the article is true. They compare the **writing style and vocabulary** of your text with the training articles. The four models can also disagree, which is a useful hint that the article is not a typical example of either class.

# 10. What the 99% really means

Scores near 99% look amazing, but a good engineer asks what the model actually learned. Two findings from our re-run matter:

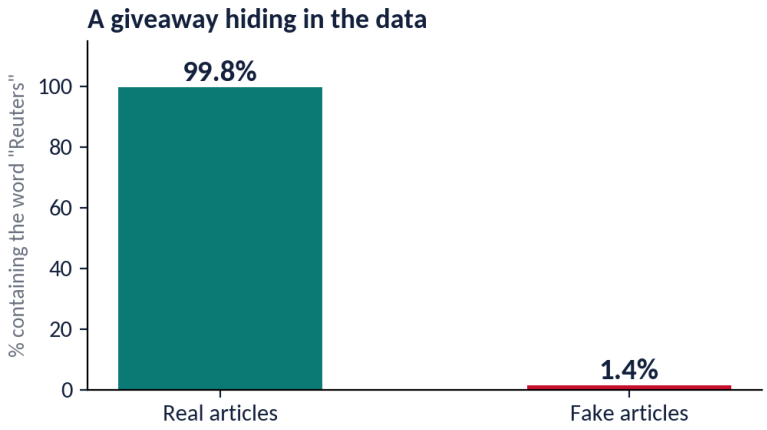



Figure 6. "Reuters" appears in almost every real article and almost no fake ones (our measurement on the files).

- **The source is a giveaway.** Since nearly every real article says "(Reuters)", the model can score well by spotting that word and other wire-service habits.
- **But it is not only Reuters.** When we removed "Reuters" from the real articles and retrained Logistic Regression, accuracy only moved from 98.63% to 98.12%. Plenty of other style clues remain, such as weekday names, "said", "via", "read" and "image".
- **Narrow data.** All articles come from a few sources and mainly cover US politics around 2015 to 2018. The test set is drawn from the same pool, so 99% describes performance on *more articles like these*, not on any news from any outlet.

**Why real news can come out as "Fake"**

If you paste a real article from a different outlet, with a different writing style or a different topic, the model sees vocabulary that looks more like the fake pool. That is exactly what happened when a real article was tested during our Colab session. It is a limitation of the data and approach, not a mistake in the code.

## Other limitations

- No fact-checking, no knowledge of the real world or of recent events.
- TF-IDF ignores word order and meaning.
- No random seed, so numbers change slightly each run.
- 630 fake articles have empty text, so the model has little to learn from them; headline text is thrown away.

## How the project could be improved

- Remove source markers (like "Reuters") and re-test; train on several datasets from different outlets and years.
- Use word pairs (n-grams) in TF-IDF, or sentence embeddings, or a transformer model such as BERT for context.
- Fix the random seed and use cross-validation for stable numbers; add a confusion matrix for each model.
- Serve the best model through a small web app, with a probability score instead of a hard label.

# 11. Small issues in the notebook (and fixes)

| Where | Issue | Suggested fix |
|---|---|---|
| Decision Tree report | Prints classification_report(y_test, pred_lr), so it shows Logistic Regression results. | Use pred_dt. |
| wordopt() | re.sub('<.*?>+', b'', text) uses a bytes replacement with a string pattern. It only works because the line never matches (all "<" were removed earlier). | Use '' instead of b'', or move this line before the \W replacement. |
| train_test_split | No random_state, so results change each run. | Add random_state=42 (and optionally stratify=y). |
| Imports | matplotlib, numpy, seaborn, accuracy_score are never used. | Remove them, or add charts using them. |
| Hold-out rows | The 20 held-back articles are created but never fed to the models. | Use them as an extra sanity check. |
| Data | Source markers such as "Reuters" stay in the text. | Strip them before cleaning to make the task harder and fairer. |

# 12. Questions you may be asked

**Q. What problem does this project solve?**

It classifies a news article as fake or real from its text, using supervised machine learning.

**Q. Why did you use TF-IDF instead of just counting words?**

TF-IDF down-weights words that appear everywhere and up-weights distinctive ones, so common words like 'the' do not dominate.

**Q. Why split into training and test sets?**

To measure performance on articles the model has never seen. Testing on training data would hide overfitting.

**Q. Why is the vectorizer fitted only on the training data?**

Otherwise information from the test set leaks into the model and the test result becomes too optimistic.

**Q. Why shuffle the data?**

After merging, all fake articles come first. Without shuffling, the split could produce a test set with only one class.

**Q. Why did you drop the title, subject and date columns?**

Only the text is used as the input. The subject column in particular would give the answer away, since its values differ between the two files.

**Q. Which model is best?**

All four are around 98.7% to 99.5%. The differences are small and vary with the random split, so we would pick by speed and interpretability: Logistic Regression is fastest and easiest to explain.

**Q. Is 99% accuracy realistic for real-world news?**

No. The data comes from a few sources and a narrow period, and the real articles share Reuters-style wording. The model partly learns style and source, so performance on other outlets would be lower.

**Q. Can the model check whether a claim is true?**

No. It only compares writing patterns with its training articles. It does not know facts.

**Q. What would you improve?**

Use more diverse datasets, remove source markers, try n-grams or a transformer model, fix the random seed, and add cross-validation.

# 13. Quick recap: running it on Google Colab

```
!git clone https://github.com/kapilsinghnegi/Fake-News-Detection.git
%cd /content/Fake-News-Detection
!unzip -o Datasets/datasets.zip -d Datasets
!find Datasets -name "*.csv" -exec cp {} . \;
# now open the notebook and use Runtime > Run all
```

The decision tree and gradient boosting cells are the slowest on 95,000 features; allow several minutes for the full run.